

轻映 QingYing 《技术选型与架构说明书》

1. 文档概述

1.1 项目名称

轻映 QingYing

1.2 项目定位

轻映是一款面向 Windows 的轻量级本地截图工具，以 **C++17 + Win32 原生接口** 实现，不依赖 Qt、Electron 等重型 GUI 框架，以绿色单文件形式发布。

产品核心路径为：

热键 → 框选 → 标注 → 复制 / 保存

在此基础上提供：

- 窗口吸附
- 多枚钉图
- 长截图
- 自然语言口令
- Agent / Skill / MCP 调用

产品要求主路径保持本地闭环，模型不进入安装包。核心轻量指标为发布体积 ≤ 20 MB、常驻内存 ≤ 40 MB、热键至遮罩 ≤ 300 ms。

2. 技术目标与设计原则

2.1 技术目标

指标	
发布体积	
常驻内存	
热键至遮罩	≤
主路径	约 1
支持系统	\
发布形态	绿
GUI 框架	不使用 Qt /
本地截图	
本地口令	
云端 AI	
MCP / Skill	F9
网络依赖	F1~F

2.2 设计原则

原则一：主路径本地优先

普通截图、标注、复制、保存、钉图和长截图均由本地代码完成。

Plain Text

- 1 热键
- 2 ↓
- 3 QingYing
- 4 ↓
- 5 本地截图引擎
- 6 ↓
- 7 本地处理
- 8 ↓
- 9 剪贴板 / 文件 / 钉图

不允许因为增加 AI 能力而让主截图流程依赖网络。

原则二：模型只负责“理解”，不负责“执行”

自然语言路径采用：

Plain Text

1 用户自然语言

2 ↓

3 本地口令表

4 ↓

5 标准 Action

6 ↓

7 本地执行

本地无法匹配时：

Plain Text

1 用户自然语言

2 ↓

3 云端 LLM（可选）

4 ↓

5 标准 Action

6 ↓

7 本地执行

模型不读取屏幕、不参与截图、不参与长截图拼接。

原则三：Agent 调用已有能力，不重新实现截图

MCP / Skill 不建立第二套截图引擎。

Plain Text

1 GUI

2 |

3 自然语言

4 |

5 MCP

6 |

7 ▼

8 Action Dispatch

9 |

10 ▼

11 Capture / Crop / LongShot / Save

这样可以避免 GUI、口令、MCP 各维护一套截图实现。

原则四：功能模块化，但不引入重型框架

模块之间通过明确接口连接，避免因功能增加导致主程序无限膨胀。

3.技术选型

3.1 总体技术栈

技术领域	选型	主要原因
开发语言	C++17	控制体积、性能和 Win 生能力
GUI	Win32	避免 Qt / Electron 运行
窗口绘制	Win32 + Direct2D / GDI	满足遮罩、标注、钉图需求
屏幕捕获	DXGI Desktop Duplication	高效获取屏幕帧
兼容捕获	GDI	对特殊窗口或兼容场景 fallback
多显示器	Win32 Monitor API	获取显示器及 DPI 信息
DPI	Per-Monitor DPI Awareness	正确处理 125% / 150%
剪贴板	Win32 Clipboard API	系统原生复制
全局热键	RegisterHotKey	系统级热键
单实例	Named Mutex	防止多实例运行
IPC	Named Pipe	MCP 与托盘主进程通信
MCP	内置 MCP Bridge	避免额外安装独立运行
JSON	轻量 JSON 实现	MCP / Action 参数解析
发布	单 EXE	满足绿色分发要求

3.2 为什么不使用 Qt

轻映的核心约束是：

在 Windows 上以 ≤ 20 MB 的绿色单文件提供截图能力。

使用 Qt 需要考虑 Qt Core、Qt GUI、Qt Widgets、平台插件及其他运行时依赖，会增加发布体积和单文件发布复杂度。

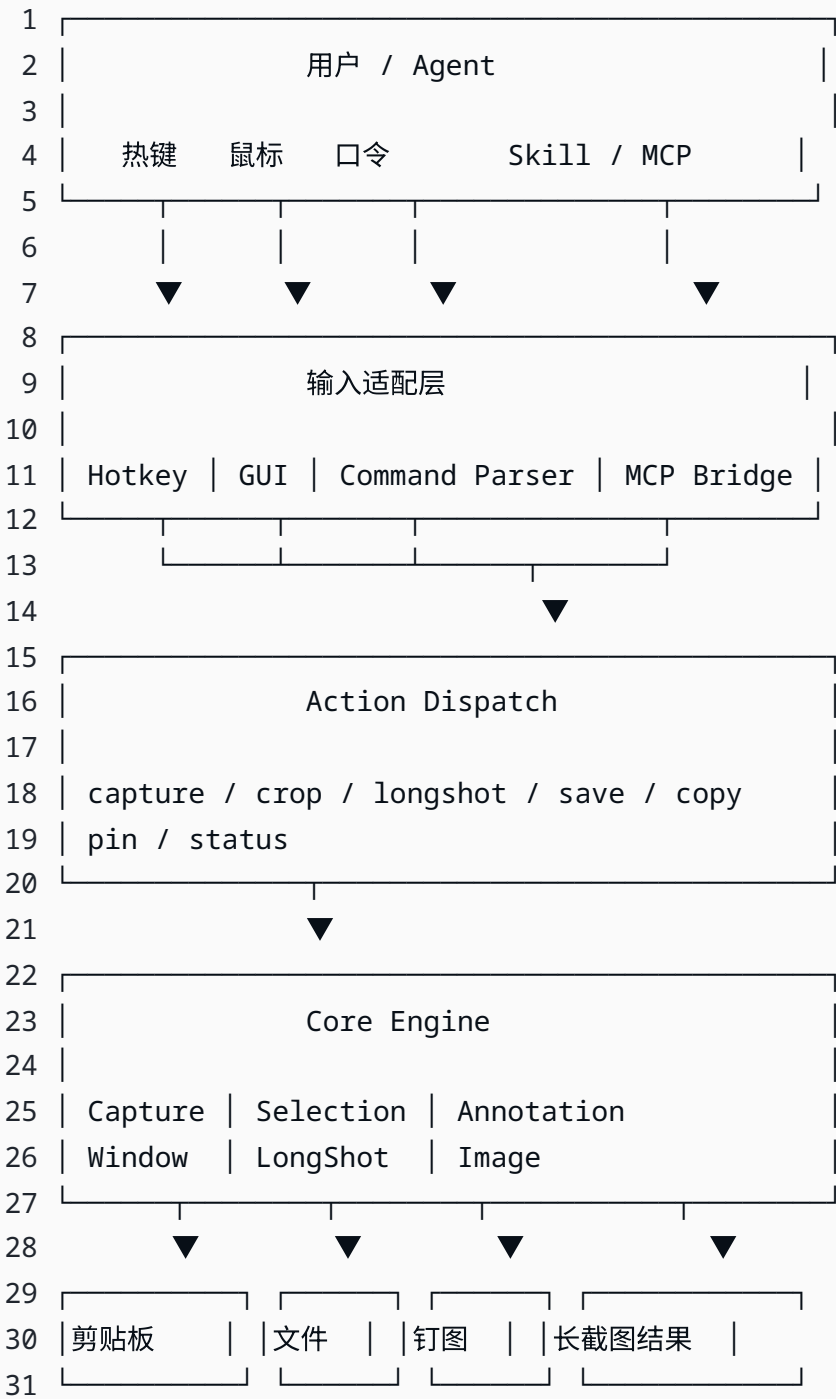
因此选择：

Win32 + Direct2D / GDI

通过原生窗口、自绘控件和系统 API 实现核心交互。

4. 总体系统架构

4.1 分层架构



5.核心模块设计

5.1 Application / Tray

负责：

- 程序启动
- 系统托盘

- 全局热键
- 单实例
- 生命周期管理
- 模块初始化
- MCP 服务状态

建议核心对象：

C/C++

```
1 class Application;  
2 class TrayController;  
3 class HotkeyManager;  
4 class SingleInstanceGuard;
```

启动流程：

Plain Text

```
1 QingYing.exe  
2   ↓  
3 SingleInstanceGuard  
4   ↓  
5 初始化 CaptureEngine  
6   ↓  
7 初始化 ActionDispatcher  
8   ↓  
9 注册全局热键  
10  ↓  
11 创建托盘  
12  ↓  
13 进入消息循环
```

5.2 屏幕捕获模块

CaptureEngine

负责：

- 全屏捕获
- 指定显示器捕获
- 窗口区域捕获

- 捕获结果转换
- 多显示器处理

接口建议：

C/C++

```
1 struct CaptureRequest {
2     enum class Type {
3         FullScreen,
4         Monitor,
5         Window,
6         Region
7     };
8
9     Type type;
10    HWND hwnd = nullptr;
11    RECT region{};
12 };
13
14 struct CaptureResult {
15     Image image;
16     RECT physicalRect{};
17     int monitorIndex = -1;
18 };
19
20 class ICaptureEngine {
21 public:
22     virtual ~ICaptureEngine() = default;
23
24     virtual Result<CaptureResult>
25     capture(const CaptureRequest& request) = 0;
26 };
```

DXGI Desktop Duplication

主捕获路径采用：

DXGI Desktop Duplication

GDI 作为兼容路径。

整体策略：

Plain Text

- 1 正常捕获
- 2 ↓
- 3 DXGI
- 4
- 5 DXGI 不适用
- 6 ↓
- 7 GDI fallback

5.3 多显示器与 DPI

产品要求在双屏、125% / 150% DPI 下正常工作。

必须明确区分：

Plain Text

- 1 逻辑坐标
- 2 ↓
- 3 Windows UI / 鼠标
- 4 ↓
- 5 物理像素
- 6 ↓
- 7 截图 Bitmap

核心原则：

捕获统一使用物理像素，UI 交互根据当前 DPI 做坐标转换。

例如：

Plain Text

- 1 鼠标逻辑坐标
- 2 ↓
- 3 DPI Scale
- 4 ↓
- 5 物理像素坐标
- 6 ↓
- 7 CaptureEngine

5.4 选区遮罩模块

SelectionOverlay

负责：

- 全屏半透明遮罩
- 鼠标框选
- 八点调区
- 窗口吸附
- Esc 取消
- 右键取消
- 工具栏

建议状态：

C/C++

```
1 enum class SelectionState {  
2     Idle,  
3     HoveringWindow,  
4     Selecting,  
5     Resizing,  
6     Moving,  
7     Annotating,  
8     Completed,  
9     Cancelled  
10 };
```

采用显式状态机，避免所有逻辑集中在鼠标事件处理中。

5.5 标注引擎

采用：

矢量对象列表 + 撤销栈

示意：

C/C++

```
1 struct Annotation {  
2     AnnotationType type;  
3     Rect bounds;  
4     Color color;  
5     float width;  
6     std::wstring text;  
7 };
```

对象模型：

Plain Text

```
1 AnnotationDocument  
2 |— Rectangle  
3 |— Ellipse  
4 |— Arrow  
5 |— Brush  
6 |— Text  
7 |— Mosaic
```

最终导出时：

Plain Text

```
1 原始截图  
2   +  
3 Annotation Objects  
4   ↓  
5 Render  
6   ↓  
7 最终 PNG
```

这样支持修改、删除和撤销。

5.6 钉图模块

每一张钉图对应一个独立窗口：

```
1 PinManager
2   |— PinWindow 1
3   |— PinWindow 2
4   |— PinWindow 3
5   |— ...
```

支持：

- 置顶
- 拖拽
- 复制
- 保存
- 关闭
- 多实例

截图时必须排除自身钉图窗口，可根据最终捕获 API 验证结果采用捕获排除策略或临时隐藏策略。

5.7 长截图模块

LongShotEngine

流程：

```
1 选择滚动区域
2      ↓
3 识别目标窗口
4      ↓
5 保存初始帧
6      ↓
7 滚动
8      ↓
9 获取下一帧
10     ↓
11 计算重叠区域
12     ↓
13 拼接
14     ↓
15 继续滚动
16     ↓
17 达到结束条件
18     ↓
19 输出长图
```

本期支持范围

严格限制：

- Windows 记事本
- 文件资源管理器
- Microsoft Edge

不承诺任意应用、任意网页、任意自绘滚动容器。

拼接算法

Plain Text

```
1 Frame N
2   ↓
3 Frame N+1
4   ↓
5 寻找重叠区域
6   ↓
7 估计 Offset
8   ↓
9 裁掉重复部分
10  ↓
11 Vertical Stitch
```

具体采用帧差、相关匹配等方案，以实际 Demo 效果确定。

6.Action Dispatch

Action Dispatch 是连接 GUI、自然语言和 Agent / MCP 的统一执行层。

6.1 请求与结果

C/C++

```
1 struct ActionRequest {
2     ActionType type;
3     JsonObject params;
4 };
5
6 struct ActionResult {
7     bool success;
8     std::string errorCode;
9     std::string message;
10    JsonObject data;
11 };
```

统一调用：

Plain Text

```
1 capture_window
2     ↓
3 ActionRequest
4     ↓
5 ActionDispatcher
6     ↓
7 WindowCapture
```

6.2 标准 Action

Plain Text

```
1 capture_window
2 crop_center
3 longshot_foreground
4 save
5 copy
6 pin
7 status
8 suggest_name
```

GUI、自然语言和 MCP 均通过 Action Dispatcher 进入本地能力层。

7.自然语言与 AI 架构

7.1 本地口令优先

Plain Text

```
1 用户输入
2     ↓
3 LocalCommandTable
4     ↓
5 标准 Action
```

例如：

Plain Text

```
1 “帮我截当前窗口”
2     ↓
3 capture_window
```

Plain Text

```
1 “帮我截中间 200×200”
2     ↓
3 crop_center
4 width = 200
5 height = 200
```

7.2 云端模型作为 fallback

Plain Text

```
1 用户输入
2     ↓
3 LocalCommandTable
4     |
5     └─ 命中 → Action
6     |
7     └─ 未命中
8         ↓
9         Cloud LLM
10        ↓
11        Standard Action
```

模型输出必须经过：

Plain Text

```
1 LLM
2 ↓
3 Action Schema Validation
4 ↓
5 Action Dispatcher
6 ↓
7 Local Execution
```


模型只解析用户输入的这一句文本，不读取屏幕，不参与截图或拼接。

7.3 建议文件名

`suggest_name()` 属于可选云端能力。

建议流程：

Plain Text

```
1 suggest_name()  
2     ↓  
3 确认允许云端建议  
4     ↓  
5 当前一张结果图  
6     ↓  
7 Cloud LLM  
8     ↓  
9 返回建议文件名
```

除用户明确需要建议文件名外，不上传当前结果图。

8.Agent / Skill / MCP 架构

8.1 定位

轻映不是一个自主操作电脑的“截图 Agent”，而是：

一个向 Agent 提供本地截图能力的 Tool Provider。

Agent 不直接控制鼠标、键盘、DXGI 或滚轮，而是通过标准 Tool 调用轻映已有能力。

Plain Text

```
1 Agent  
2     ↓  
3 MCP  
4     ↓  
5 QingYing  
6     ↓  
7 本地截图
```

8.2 为什么需要 MCP

传统 Agent 需要截图时，可能需要：

Plain Text

```
1 Agent
2 ↓
3 临时编写 Python
4 ↓
5 窗口识别
6 ↓
7 截图
8 ↓
9 裁切
10 ↓
11 保存
```

这会重复实现本地截图能力，并增加执行耗时和失败概率。

引入 MCP 后：

Plain Text

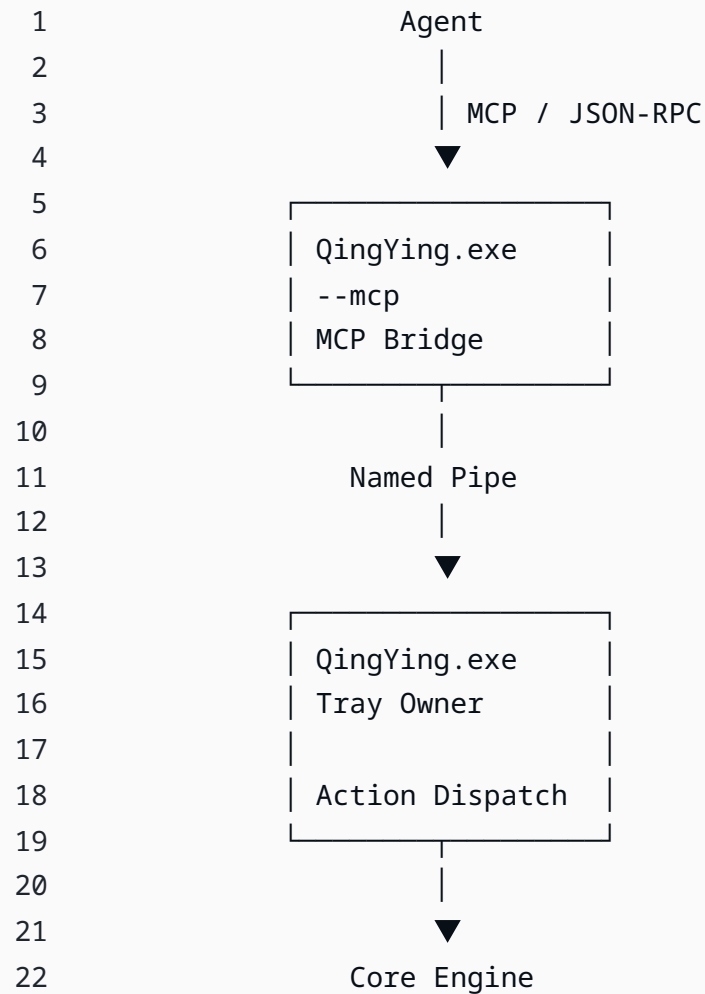
```
1 Agent
2 ↓
3 capture_window()
4 ↓
5 QingYing
6 ↓
7 截图
```

因此 MCP 的核心价值是：

把轻映已有截图能力标准化，使 Agent 可以直接调用，而不是每次重新编写截图脚本。

8.3 MCP Bridge

为了保持绿色单文件，同时满足“轻映已在托盘运行”的调用前提，建议 QingYing.exe 支持正常托盘模式和 `--mcp` Bridge 模式。



MCP Bridge 不实现截图逻辑，最终仍由主进程的 Action Dispatcher 执行。

8.4 MCP 调用流程

例如：

```
1 capture_window({
2     "query": "记事本"
3 })
```

执行：

Plain Text

```
1 Agent
2 ↓
3 MCP Bridge
4 ↓
5 Named Pipe
6 ↓
7 ActionDispatcher
8 ↓
9 WindowManager
10 ↓
11 CaptureEngine
12 ↓
13 Image
14 ↓
15 Result
16 ↓
17 MCP Bridge
18 ↓
19 Agent
```

9.MCP Tool 设计

9.1 status

Plain Text

```
1 status()
```

作用：

查询轻映是否已经运行。

示例返回：

JSON

```
1 {  
2   "running": true,  
3   "version": "1.0.0"  
4 }
```

9.2 capture_window

Plain Text

```
1 capture_window(query)
```

作用：

按窗口名查找并截图。

参数：

JSON

```
1 {  
2   "query": "记事本"  
3 }
```

如果存在多个候选窗口，不自动猜测，应返回多个候选并要求使用者选择。

9.3 crop_center

Plain Text

```
1 crop_center(width, height)
```

例如：

JSON

```
1 {  
2   "width": 200,  
3   "height": 200  
4 }
```

获取当前画面并计算虚拟屏幕中央区域。

9.4 longshot_foreground

Plain Text

```
1 longshot_foreground()
```

对当前前台页面进行长截图。

仅保证：

- 记事本
- 文件资源管理器
- Microsoft Edge

9.5 save

Plain Text

```
1 save(path, name)
```

保存当前结果图。

最终路径经过路径和文件名校验。

9.6 copy / pin

Plain Text

```
1 copy()  
2 pin()
```

分别调用 ClipboardManager 和 PinManager。

9.7 suggest_name

Plain Text

```
1 suggest_name()
```

可选云端能力。只有在用户确认后，才将当前一张结果图交给云端模型建议文件名。

10.MCP 错误处理

所有 MCP 调用统一返回结构化错误。

C/C++

```
1 enum class ErrorCode {
2     Success,
3     NotRunning,
4     InvalidArgument,
5     WindowNotFound,
6     MultipleWindows,
7     CaptureFailed,
8     LongShotUnsupported,
9     SaveFailed,
10    PermissionDenied,
11    Cancelled
12 };
```

示例：

JSON

```
1 {
2   "success": false,
3   "errorCode": "LONGSHOT_UNSUPPORTED",
4   "message": "当前窗口不属于本期支持的长截图应用"
5 }
```

要求：

失败返回错误，不自动重试。

11.Skill 设计

MCP 解决：

Agent 怎么调用。

Skill 解决：

Agent 什么时候调用、参数怎么填、失败怎么办。

11.1 Skill 内容

建议包含：

Markdown

```
1 # QingYing Screenshot Skill
2
3 ## 适用场景
4
5 需要获取 Windows 本地截图时使用。
6
7 ## Tools
8
9 capture_window
10 crop_center
11 longshot_foreground
12 save
13 copy
14 pin
15 status
16
17 ## 使用规则
18
19 1. 优先调用 status
20 2. 需要指定窗口时使用 capture_window
21 3. 不确定窗口时不要猜测
22 4. 长截图只用于支持的应用
23 5. 失败后不要循环重试
24 6. 无法调用时提示用户使用热键手动截图
```

Skill 本身不截屏、不拼图、不执行系统操作。

11.2 MCP 与 Skill 的关系

Plain Text



12.安全边界

12.1 仅接受本机调用

MCP 不开放公网端口，只允许本机调用。

12.2 MCP 不提供任意系统操作

不提供：

Plain Text

- 1 click
- 2 type
- 3 scroll(anywhere)
- 4 execute_command
- 5 read_screen

只提供已经定义的截图能力。

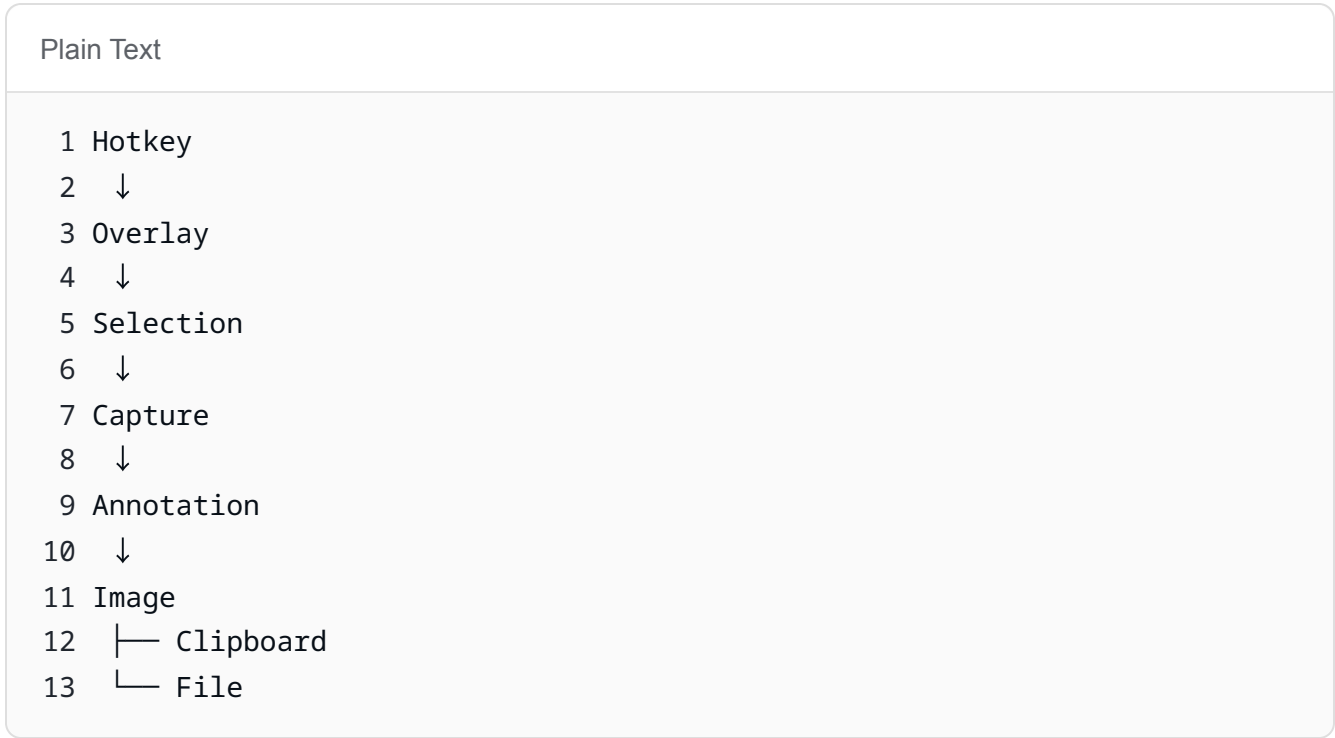
12.3 MCP 不允许模型直接访问屏幕

模型获得的是 Tool Result，而不是整个桌面画面。

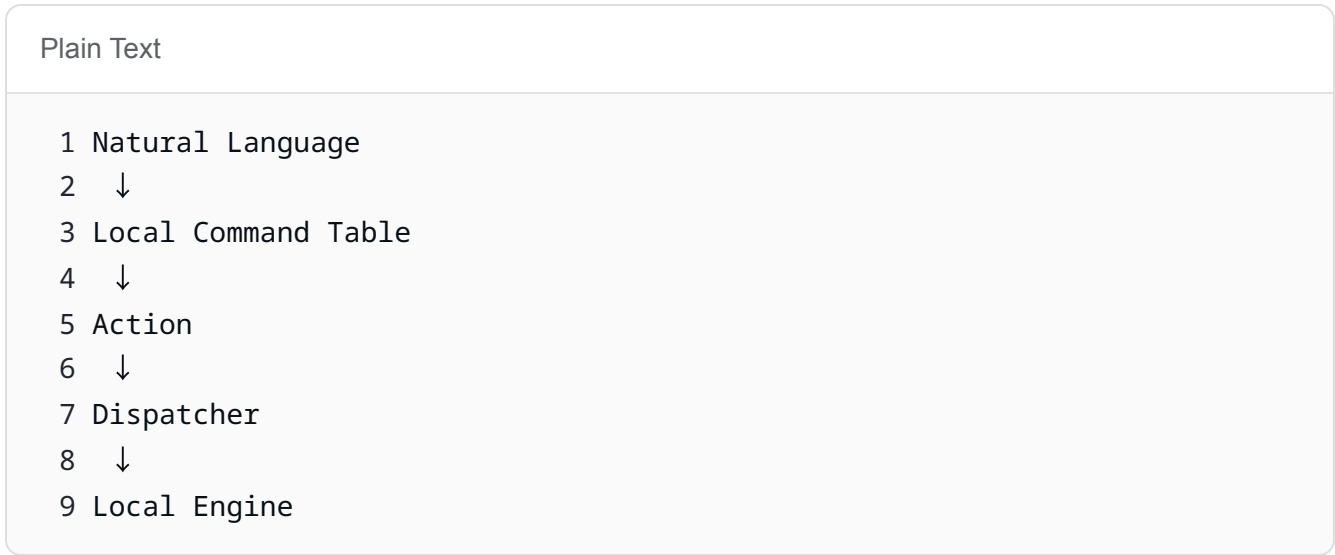
除 `suggest_name` 且用户确认外，不上传截图。

13.数据流

13.1 普通截图



13.2 自然语言



13.3 MCP

Plain Text

```
1 Agent
2 ↓
3 MCP
4 ↓
5 ActionRequest
6 ↓
7 ActionDispatcher
8 ↓
9 Local Engine
10 ↓
11 ActionResult
12 ↓
13 Agent
```

13.4 云端口令

Plain Text

```
1 Natural Language
2 ↓
3 Local Table
4 ↓
5 Miss
6 ↓
7 Cloud LLM
8 ↓
9 Structured Action
10 ↓
11 Schema Validation
12 ↓
13 ActionDispatcher
14 ↓
15 Local Engine
```

14.模块依赖关系

```
1 Application
2   |— Tray
3   |— Hotkey
4   |— MCP Bridge
5   |— Action Dispatcher
6       |
7       |— Capture Engine
8       |— Selection Engine
9       |— Annotation Engine
10      |— LongShot Engine
11      |— Pin Manager
12      |— Clipboard Manager
13      |— File Manager
```

关键原则：

MCP Bridge 依赖 Action Dispatcher，而不是直接依赖 CaptureEngine。

这样 GUI、自然语言和 MCP 都使用同一套业务能力。

15. 关键接口

建议定义统一接口：

C/C++

```
1 class IActionHandler {
2 public:
3     virtual ~IActionHandler() = default;
4
5     virtual ActionResult
6     execute(const ActionRequest& request) = 0;
7 };
```

Dispatcher:

C/C++

```
1 class ActionDispatcher {
2 public:
3     ActionResult dispatch(const ActionRequest& request);
4
5 private:
6     std::unordered_map<ActionType,
7                         std::unique_ptr<IActionHandler>> handlers_;
8 };
```

处理器可以拆分为：

Plain Text

```
1 CaptureWindowHandler
2 CropCenterHandler
3 LongShotHandler
4 SaveHandler
5 CopyHandler
6 PinHandler
7 StatusHandler
8 SuggestNameHandler
```

16.性能与资源设计

16.1 ≤ 300 ms 热键响应

关键路径：

Plain Text

```
1 Hotkey
2 ↓
3 唤醒
4 ↓
5 创建 Overlay
6 ↓
7 显示遮罩
```

不在该关键路径执行：

- 云端请求
- 图片压缩
- 长截图初始化
- MCP 初始化
- 模型加载

16.2 常驻内存 ≤ 40 MB

托盘状态下只保持：

Plain Text

- 1 Tray
- 2 Hotkey
- 3 ActionDispatcher
- 4 CaptureManager
- 5 基本资源

大型 Bitmap 按需创建，任务结束后释放。

16.3 体积 ≤ 20 MB

控制手段：

- 不使用 Qt
- 不使用 Electron
- 不打包 Python Runtime
- 不打包模型
- 控制第三方库
- 优先使用系统 API
- Release 编译
- 删除无关资源

17.异常与降级策略

17.1 无网络

Plain Text

```
1 F1～F7
2 ↓
3 正常使用
4
5 F8
6 ↓
7 本地口令
8 ↓
9 正常使用
10
11 云端解析
12 ↓
13 不可用
14 ↓
15 提示未匹配
```

17.2 无云密钥

与无网络相同，不影响 F1～F7。

17.3 MCP 未启动

Plain Text

```
1 MCP
2 ↓
3 status
4 ↓
5 NotRunning
```

Agent 应提示用户先启动轻映托盘程序。

17.4 长截图不支持

```
1 longshot_foreground
2 ↓
3 检查前台窗口
4 ↓
5 非记事本 / Explorer / Edge
6 ↓
7 LONGSHOT_UNSUPPORTED
```

不自动尝试任意应用。

18.关键技术验证

18.1 多显示器 / DPI

验证：

- 单屏
- 双屏
- 主副屏不同 DPI
- 125%
- 150%

检查：

- 选区位置
- 截图尺寸
- 窗口吸附
- 钉图位置

18.2 长截图

分别验证：

- 记事本 ≥ 1 次
- 文件资源管理器 ≥ 1 次
- Edge ≥ 1 次

检查：

- 滚动是否正常

- 是否漏帧
- 是否重复
- 拼接边界是否明显

18.3 钉图

至少验证 2 枚同时存在：

- 可拖拽
- 可复制
- 可关闭
- 新截图不包含 PinWindow

18.4 MCP

建议验证：

Plain Text

```
1 status()  
2 capture_window()  
3 crop_center()  
4 save()  
5 copy()  
6 pin()  
7 longshot_foreground()
```

`suggest_name()` 不作为无云验收的必须项。

19.技术风险

风险	等级	应对
DPI 坐标错误	高	统一物理像素 + DPI 转
长截图拼接失败	高	限定三类应用
自身钉图被截进去	中	捕获前排除 / 临时隐藏
Win32 自绘复杂	中	模块化控件
MCP 与主进程通信	中	Named Pipe
MCP 参数异常	中	Schema + Action Valic
云端不可用	低	本地口令 fallback
体积超标	中	禁止重型框架和模型进包
内存超标	中	Bitmap 按需分配

20.开发优先级

```
1 P0
2 |
3 |— 单实例
4 |— 托盘
5 |— 全局热键
6 |— 全屏遮罩
7 |— 框选
8 |— CaptureEngine
9 |— Clipboard
10 |— Save
11     ↓
12 P1
13 |
14 |— 调区
15 |— 标注
16 |— 窗口吸附
17 |— DPI / 多屏
18     ↓
19 P2
20 |
21 |— Pin
22 |— LongShot
23     ↓
24 P3
25 |
26 |— Local Command
27 |— Action Dispatcher
28 |— MCP
29     ↓
30 P4
31 |
32 |— Skill
33 |— Cloud LLM fallback
```

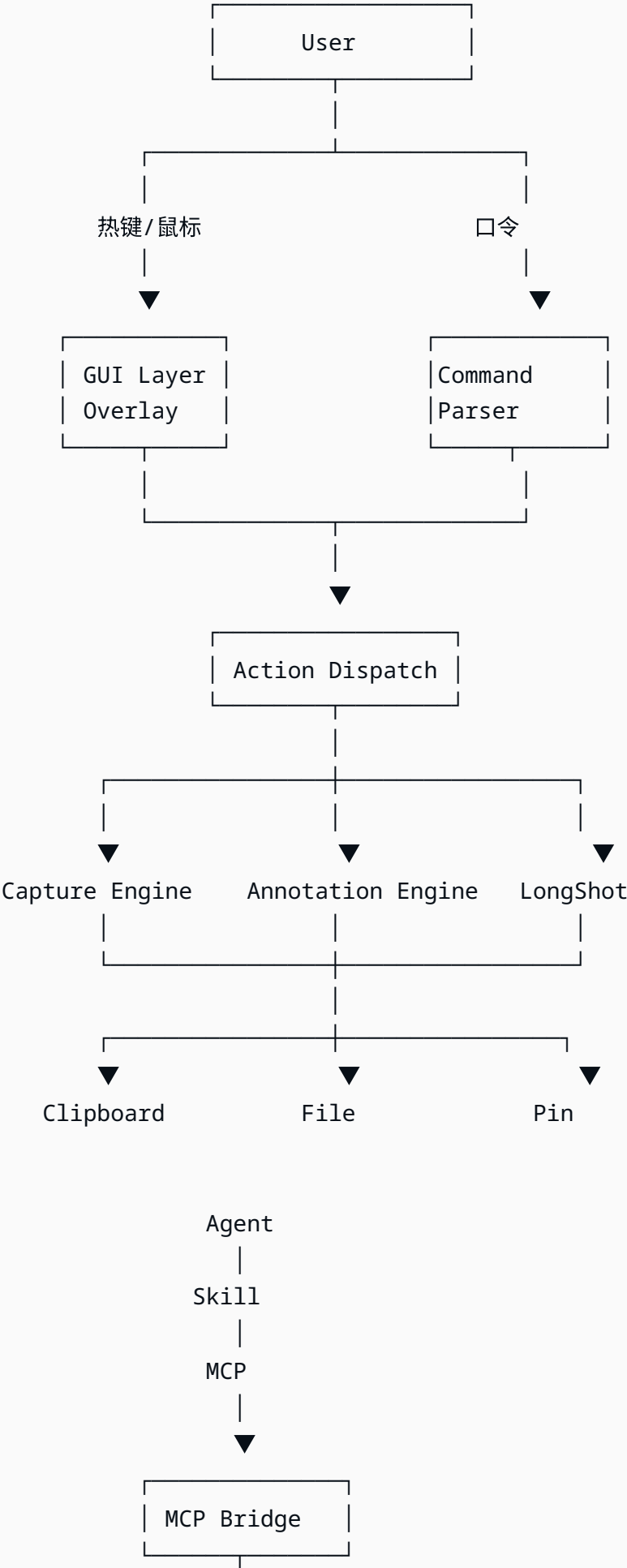
需要注意：

Action Dispatcher 应在 F1~F4 阶段建立，而不是等 MCP 阶段再设计。

这样 GUI、口令和 MCP 后续都可以复用同一套标准 Action。

21.最终架构图

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43





22.最终技术架构结论

轻映采用 **C++17 + Win32 + DXGI Desktop Duplication + GDI/Direct2D** 的原生 Windows 技术路线，在不引入 Qt、Electron、Python Runtime 和本地模型的前提下，实现绿色单文件截图工具。

架构采用：

输入适配层 → Action Dispatch → 本地能力引擎

统一承载 GUI、自然语言和 Agent 三种调用方式。

其中：

- **GUI**：解决人直接截图；
- **本地口令**：解决人通过自然语言截图；
- **Skill**：描述 Agent 如何使用截图能力；
- **MCP**：提供标准 Tool；
- **Action Dispatch**：统一执行；
- **Capture / Annotation / Pin / LongShot**：完成实际本地能力。

因此，Agent 不直接控制屏幕，而是：

通过 MCP 调用轻映已经具备的本地截图能力。

这样既能解决 Agent 每次从零编写 Python 截图脚本的问题，又不会改变轻映“轻量本地截图工具”的核心定位。

附录 A：F1～F9 与技术模块对应关系

功能	技术模块
F1 自定义截图区域	SelectionOverlay + CaptureEngine
F2 窗口吸附	WindowDetector + SelectionOverlay
F3 截图标注	AnnotationEngine
F4 导出	ClipboardManager + FileManager
F5 钉图	PinManager
F6 长截图	LongShotEngine
F7 托盘与热键	TrayController + HotkeyManager
F8 口令截图	CommandParser + ActionDispatcher
F9 Agent / MCP	MCP Bridge + Named Pipe + ActionDispatcher

附录 B：F9 标准封装清单

Tool	作用	是否依赖云
<code>status()</code>	查询轻映是否运行	否
<code>capture_window(query)</code>	按窗口名截图	否
<code>crop_center(width,height)</code>	截取中央指定尺寸	否
<code>longshot_foreground()</code>	当前前台页面长截图	否
<code>save(path,name)</code>	保存当前结果	否
<code>copy()</code>	复制当前结果	否
<code>pin()</code>	钉住当前结果	否
<code>suggest_name()</code>	建议文件名	是，可选

附录 C：本期明确不做

- 整屏或逐帧提交给模型查看
- 标注与调区等指针操作的 MCP 自动化
- DXGI 等内部接口直接暴露给 Agent
- 任意应用长截图
- 自动点击

- 任意应用自动滚动
- 截图 Agent
- 云同步图床
- 录屏 / GIF 产品化
- 账号体系
- 跨平台客户端
- 本地模型打包进安装程序